

I. PRECISIONS SUR LES FONCTIONS

Video Fonctions

Contenu :

1. Intérêt des fonctions : factorisation de code >8 minutes
2. Flexibilité du passage de paramètres
 - a. Optionnels, par défaut >10 min
 - b. dans un ordre quelconque : >12 min
 - c. en nombre indéfini parExemple(*parametres) : 16 min< <21 min
3. Retours de fonctions > 21 min
4. Intérêt de la factorisation : > 25 min

1. PASSAGE DE PARAMETRES :

Exo 1. Déterminer toutes les réponses de la console à l'exécution du code ci-dessous :

```
def un_calcul1(a,b,c):
    return (a+(b*c))

print("calcul #1 : ", un_calcul1(10,20,30))

def un_calcul2(a,b,c=300):
    return (a+(b*c))

print("calcul #2a : ", un_calcul2(10,20,30))
print("calcul #2b : ", un_calcul2(10,20))

def un_calcul3(a=100,b=200,c=300):
    return (a+(b*c))

print("calcul #3a : ", un_calcul3(b=20))
print("calcul #3b : ", un_calcul3())
print("calcul #3d : ",
      un_calcul3(c=10,a=20,b=30))
print("calcul #3e : ", un_calcul3(c=20,b=30))

def un_calcul4(*par):
    s=0
    for a in par:
        s=s+a
    return s

print("calcul #4a : ", un_calcul4(1,2,3))
print("calcul #4b : ", un_calcul4(2,3,1))
print("calcul #4d : ", un_calcul4(123))
print("calcul #4e : ", un_calcul4(3,2,1,-1,-2))
print("calcul #4e : ", un_calcul4(3,-2))
print("calcul #4f : ", un_calcul4())
```

2. FONCTIONS LAMBDA



Cf programmation fonctionnelle

II. MODULES, INTERFACES ET ENCAPSULATION



La nécessaire factorisation du code (pour des raisons de facilité de debug, maintenabilité, réutilisabilité, ...) amène à plus grande échelle à la modularité du code :

La modularité est un concept qui implique

- ✚ de structurer un ensemble logiciel en composants les plus indépendants possibles (**couplage faible**) afin de faciliter le développement de gros projets, et d'en augmenter la lisibilité et la réutilisabilité
- ✚ de spécifier les liens entre les modules en mettant à disposition d'un développeur utilisateur l'**interface du module** (arguments attendus de chaque fonction, spécification des retours de chaque

fonction mise à disposition) tout en masquant les détails de l'implémentation de chaque fonction : c'est la mise en œuvre du principe d'**encapsulation**.



[Video Modularité](#)

Fonction lambda (< 7 min)

Un module est un **fichier.py** contenant des fonctions utilisables dans le fichier python importateur

Ex 1 : le module **TKinter** (ToolKit Interface) pour une bibliothèque d'outils d'interface graphique ou le module **turtle** (dessin en mode tortue)



Pour importer la fonction **bonjour()** du module **mon_module.py**

- **import mon_module**

la fonction est alors appelée dans le fichier importateur par **mon_module.bonjour(...)**

Encapsulation : Il suffit alors d'en connaître l'interface pour l'utiliser. Le code qui l'implémente est masqué

- si le module est dans le sous-dossier « mon_sous_dossier » :
import mon_sous_dossier.mon_module
la fonction est alors appelée **mon_sous_dossier.mon_module.bonjour()**
 - Dans le cas d'un alias :
import mon_sous_dossier.mon_module as modul
la fonction est alors appelée **modul.bonjour()**
- **from mon_module import bonjour()**
la fonction est alors appelée dans le fichier importateur par **bonjour(...)**
 - Pour importer **toutes** les fonctions du module **mon_module** :
from mon_module import *
Les fonctions importées n'ont alors pas besoin d'être préfixées par le nom du module.



Le contenu d'un module s'obtient avec la fonction **dir()**:

```
>>> import timeit
>>> dir(timeit)
['Timer',
 '__all__',
 '__cached__',
 ...,
 '__package__',
 '__spec__',
 ...,
 'template',
 'time',
 'timeit']
```

- Un package est un **ensemble de dossiers et sous-dossiers** contenant les fichiers python importés.

Ex 2 : Le package **matplotlib** (bibliothèque graphique) qui peut contenir des modules comme **pylab.py** ou **pyplot.py**

- Une bibliothèque est en général une collection de modules (qui peuvent être un unique module ou en package)

Ex 3 : **PIL** (bibliothèque de traitement d'image) ou **pygame** (bibliothèque de jeu en 2D)

III. TESTS

La réutilisabilité d'un module nécessite avant sa mise à disposition aux développeurs utilisateurs un niveau de tests suffisant. On parle alors de tests unitaires.

a) Tests précisés dans le docstring

Le contenu des tests et les résultats attendus de ces tests peuvent faire l'objet d'un docstring dans les fonctions qui le composent.

Ex 4 : Dans un module de tri :

```
def estTrie(t):
    """ici c'est le docstring
       retourne True si le tableau t est trié dans l'ordre croissant, False sinon
    >>> estTrie([17, 28, 39, 54, 59, 72]) #True
    >>> estTrie([17, 28, 9, 54, 59, 72])  #False
    >>> estTrie([1,1,1])                  #True
    >>> estTrie([])                       #True
    """
    for i in range(len(t)-1):
        if t[i]>t[i+1]:
            print("liste non triée")
            return False
    #print("liste triée")
    return True
```

Rq1. Ecrire le **docstring** et penser les tests **en amont du codage** est un gage de qualité du code produit ensuite.

b) Tests unitaires intégrés dans le module



avec la condition

```
if __name__ == '__main__':
```

...

(cf la [Video Modularité](#))

c) Un exemple de fichier avec tests unitaires

Exo 2. On donne le code suivant : [Tests_Tri.py](#)

```
def insere(t, i, v):
    """insère v dans t[0..i] supposé trié"""
    j = i
    while j > 0 and t[j-1] > v:
        #5 while j > 0 and t[j] > v:
            t[j] = t[j - 1]
            j = j - 1
    t[j] = v
    #9 t[j]=0

def tri_insertion(t):
    """trie le tableau t dans l'ordre croissant"""
    for i in range(1, len(t)):
        insere(t, i, t[i])
    print(t)
```

```
def test(t):
    print(t)
    tri_insertion(t)
    for i in range(len(t)-1):
        assert t[i]<t[i+1]
    print("test ok")

if __name__ == '__main__':
    test([72, 39, 28, 59, 17, 54])
    test([2])
    test([])
```

- a. Vérifier en décommentant la ligne #5 et en commentant la précédente que le bug provoqué est mis en évidence.
Expliquez la stratégie de tests adoptée pour ce module.

Il s'agit ici de tests en boîte noire : la stratégie de tests est indifférente à la manière dont la fonction est codée. Les tests sont construits à partir uniquement de l'interface spécifiée de la fonction sans se préoccuper de l'implémentation.

- b. Décommenter la ligne 9.
Commenter le fonctionnement du programme constaté et des tests.
Les tests paraissent-ils garantir un niveau de confiance suffisant ?
Proposer des stratégies de tests qui complèteraient les tests déjà implémentés.
- c. Implémenter.
(Une fonction **occurrences(t)** qui renvoie le dictionnaire des occurrences de **t** et une fonction **identiques(d1,d2)** qui renvoie un booléen précisant si les deux dictionnaires sont identiques ou non et une fonction **test(t)** qui teste le tri du tableau **t**.)



[Tests Tri -- Complet.py](#)

IV. GESTION DES ERREURS – TRAITEMENT DES EXCEPTIONS



[Video Gestion des Erreurs](#)

Contenu :

- Gestion des erreurs avec try :... except...except ...else : finally :...
- Levée d'exceptions avec **raise**
- Assertion avec **assert**

Une **exception** est un événement qui se produit pendant l'exécution d'un programme et qui perturbe le flux normal des instructions du programme. En général, lorsqu'un script Python rencontre une situation qu'il ne peut pas gérer, il déclenche une exception. Une exception est un objet Python qui représente une erreur.

Exo 3. Provoquer dans un programme les erreurs (déjà rencontrées) suivantes :

- ZeroDivisionError
- TypeError
- IndexError
- NameError
- KeyError
- AssertionError
- ValueError



[quelques erreurs.py](#)

Rq 2. Les types permettent de caractériser les opérandes ou paramètres qui sont ou non acceptables pour certaines opérations. L'utilisation de valeurs incompatibles avec une opération donnée lève une exception **TypeError** :

```
>>> 'nsi'+2
Traceback (most recent call last):
  File "<string>", line 449, in runcode
  File "<interactive input>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'

>>> 'nsi'+str(2)
'nsi2'
```

Ex 5 : Les types compatibles avec l'opérateur +

EXEMPLE	TYPE DES OPERANDES	EFFET	TYPE DU RÉSULTAT
1+2	int (1 et 2)	addition	int
1.2+3.4			
1+3.4			
"nsi"+"2"			
(1,2)+(3,4)			
[1,2]+[3,4]			

3. GESTION DES EXCEPTIONS



En Python, les exceptions peuvent être gérées à l'aide de l'instruction « try ».

Une opération critique pouvant générer une exception est placée dans le bloc « **try** » et le code qui gère cette exception est écrit dans le bloc « **except** ».

Exo 4. Quel est le résultat produit par les codes ci-dessous ?

1.

```
try :
    file = open("file.txt", "w")
    file.write("Hello World!")
except IOError:
    print("Erreur: impossible de trouver le fichier")
else:
    print ("Contenu écrit dans le fichier avec succès")
    file.close()
print (« terminé »)
```

2.

```
file = open("file.txt", "r")
file.write("Hello World!")
```

3.

```
try :
    file = open("file.txt", "r")
    file.write("Hello World!")
except IOError:
    print("Erreur: impossible d'écrire dans le fichier")
else:
    print ("Contenu écrit dans le fichier avec succès")
    file.close()
print (« terminé »)
```

4. GENERATION D'EXCEPTIONS

Il est possible de générer des exceptions de plusieurs manières à l'aide de l'instruction **raise**. C'est en particulier utile pour garantir l'utilisation de fonctions dans le domaine défini par leur interface. On prévient ainsi la propagation de bugs à l'extérieur du module incriminé, renforçant ainsi l'encapsulation des fonctions importées.

Exo 5. Quel est le résultat produit par les codes ci-dessous ?

1.

```
def saisir_age() :
    age =int(input())
    if age > 0:
        #Faire quelque chose.
    else:
        raise ValueError("Valeur inattendue!", age)
```

2. On donne le code ci-dessous :

```
def valider_data(d):
    L=[0]*3
    if d >= 0:
        L[d]=1
        print("inverse=",1/d)
    else:
        raise ValueError("erreur grossière!", d)
```

```
def saisir_entree():

    e= int(input("donner une entrée"))
    try:
        valider_data(e)
    except IOError :
        print("message 1")
    except ValueError :
        print("message 2")
    except TypeError :
        print("message 3")
    except IndexError :
        print("message 4")
    except ZeroDivisionError :
        print(« message 5 »)
    else :
        print("message 100")
    print("fin")
```

```
saisir_entree()
```

3. Quel est le comportement de l'interpréteur Python et quels sont les affichages console si l'entrée saisie vaut :

- a. 2
- b. 12
- c. zéro
- d. 0
- e. « 12 »
- f. -4

Exo 6. On donne une interface minimale pour une structure de dictionnaire :

FONCTION	DOCSTRING
cree()	crée et renvoie un dictionnaire vide
cle(d,k)	renvoie True si et seulement si le dictionnaire d contient la clé k
lit(d,k)	renvoie la valeur associée à la clé k dans le dictionnaire d ; et None si la clé k n'apparaît pas.
ecrit(d,k,v)	ajoute au dictionnaire d l'association entre la clé k et la valeur v , en remplaçant une éventuelle association déjà présente par k .

On propose de réaliser cette interface de dictionnaire avec un tableau de tableaux [**clé,valeur**], en faisant en sorte qu'aucune clé n'apparaisse plus d'une fois.

- Ecrire un module implémentant cette interface.
- La description de l'une des quatre fonctions de cette interface ne correspond pas tout à fait à l'opération équivalente des dictionnaires Python. Laquelle ?
Mettre en évidence cette différence par un jeu de de test adapté.
Corriger la description de l'interface pour se rapprocher de celle de Python et adapter l'implémentation.

5. ANNEXE : SYNTHESE DE LA SYNTAXE :

